

The Fidelity–Safety Gap in Neural Wikidata Constraint Repair

Miguel Vázquez @ ORCID

Vienna University of Economics and Business (WU Wien), Vienna, Austria

Axel Polleres @ ORCID

Vienna University of Economics and Business (WU Wien), Vienna, Austria

Abstract

Collaborative knowledge graphs such as Wikidata use property constraints to flag statements for curation. Detecting a violation, however, does not determine its repair: multiple edits may satisfy the same constraint while differing in curator agreement, collateral effects, disruption, and retention of the base statement. Historical repairs provide supervision, but imitating them need not yield constraint-satisfying repairs. We study this gap between validation and learned repair imitation.

We represent each repair case as a violation-centred graph and augment a graph neural network (GNN) with explicit nodes for local constraints. These *executable constraint factors* combine symbolic validators with neural satisfaction modules and message channels that enable inference-time interventions.

Using a corpus of 955,405 Wikidata repair instances, we evaluate all reported systems on a held-out test split of 143,316 instances. The Direct-Factor GNN reaches 0.6788 Micro-F1 for strict agreement with historical statement edits, compared with 0.5471 for the Direct-Passive GNN. Masking constraint messages lowers Micro-F1 by 0.3985 and changes 58.6% of predictions, mostly through constraints other than the origin-

ally flagged one.

Separately, symbolic post-edit evaluation shows that Candidate-DP, which applies direct candidate-level penalties, raises the resolved share among primary constraints that local validators find violated before repair from 62.78% to 77.07%, while reducing new violations in previously satisfied neighbouring constraints from 0.48% to 0.39%. These gains come at a substantial cost: historical-agreement Micro-F1 falls from 0.6788 to 0.4370, and the share of initially checkable, violated primary constraints that are fixed without deleting the base statement falls from 10.53% to 3.50%. More starkly, Candidate-SR, a diagnostic reranker trained to maximise local constraint satisfaction, predicts a deletion in every test instance and removes the base statement in 97.41% of them. Its historical-fidelity Micro-F1 is only 0.2639. This learned satisfaction-by-deletion shortcut shows that constraint satisfaction can reward removing evidence rather than useful correction.

Executable constraints improve inspectability but do not guarantee useful repairs: historical fidelity, post-edit constraint satisfaction, collateral effects, disruption, and base-statement preservation must remain distinct evaluation criteria.

2012 ACM Subject Classification Information systems → Data cleaning; Information systems → Resource Description Framework (RDF); Computing methodologies → Neural networks; Theory of computation → Logic and verification

Keywords and phrases Wikidata, Knowledge Graph Repair, Property Constraints, Graph Neural Networks, Neuro-Symbolic Learning, Data Quality, Repair Evaluation

Related Version This article builds on “Structure is the Signal” (ESWC 2026), introducing executable constraint factors and a multi-criteria evaluation of repair fidelity and symbolic safety.

Supplementary Material Detailed result tables are provided in the supplementary material; implementations, experiment specifications, and machine-readable results are available in the project repository.

Funding This research was funded in whole or in part by the Austrian Science Fund (FWF) 10.55776/COE12.

Received Date of submission **Accepted** Date of acceptance **Published** Date of publishing

Editor TGDK section area editor

1 Introduction

Collaborative knowledge graphs (KGs) such as Wikidata evolve through community edits and accumulate inconsistencies [11, 31, 5, 23, 24, 25, 27]. Wikidata property constraints describe expected property use, but flag violations for review instead of rejecting edits [33, 18, 32]. Validation therefore identifies a non-conforming pattern without selecting its repair.

A violation may be addressed by adding missing evidence, replacing a value, removing conflicting context, or deleting the statement associated with the violation (the base statement) [4, 2, 7]. Historical edits offer supervision for this choice [21, 22, 30], yet agreement with curators neither guarantees post-edit constraint satisfaction nor prevents collateral violations. Conversely, satisfaction alone cannot determine usefulness: deleting the statement that witnesses a violation may satisfy its checker, whether or not that deletion is justified.

Recent work shows that local graph structure is a strong signal for predicting historical Wikidata repairs with multi-relational graph neural networks [30]. We investigate whether the applicable rules can also be made explicit within that representation. For each repair case, we construct a violation-centred graph and represent every supported locally applicable property constraint as a node connected to the statements and entities within its scope. Each such node is associated with a symbolic validator, neural heads that estimate pre- and post-edit constraint satisfaction, and explicit message channels through which learned constraint information can influence repair prediction. We call these nodes *executable constraint factors*. They are executable because their associated validators can be evaluated on concrete graph states; this designation does not assume that the corresponding neural modules implement sound symbolic reasoning.

This representation has two roles. First, it gives the repair model explicit access to the supported constraints that apply to the local graph. Second, it makes the influence of those constraints inspectable. We compare the outputs of the satisfaction heads with the symbolic satisfaction labels used for supervision and selectively mask constraint messages at inference time to test whether predictions depend on those channels. These analyses can show that the model uses the factor pathway and that its heads align with those labels, but they cannot establish that it has learned the intended constraint semantics or that the resulting repairs are useful.

We therefore evaluate repair quality along several separate dimensions rather than reducing it to a single score. These dimensions include exact agreement with historical curator edits, repair of the originally violated constraint, post-edit satisfaction of local constraints, improvements and regressions in other constraints, edit disruption, and preservation of the base statement that triggered the violation. Using a corpus of 955,405 repair instances, we evaluate all reported systems on a held-out test split of 143,316 instances. The Direct–Factor GNN reaches 0.6788 Micro-F1, compared with 0.5471 for the Direct–Passive GNN. Masking factor pressure messages lowers its F1 by 0.3985 and changes 58.6% of predictions, mostly through non-primary factors.

Candidate–DP (Direct Penalty) resolves 77.07% of primary constraints that local validators find violated before repair, compared with 62.78% for the Direct–Factor GNN, and slightly reduces secondary regressions. However, Micro-F1 drops by 0.2418, while the rate of base-statement-preserving primary fixes falls from 10.53% to 3.50%. Candidate–SR (Satisfaction Reranker), trained only to maximise local satisfaction, predicts a deletion in every test case and removes the base statement 97.41% of the time, approaching an always-delete strategy. This *satisfaction-by-deletion* result does not make deletion universally undesirable; it shows why satisfaction must be assessed with fidelity, collateral effects, disruption, and base-statement preservation.

Research questions.

We organise the study around five questions.

■ **Table 1** Relationship to the ESWC 2026 conference paper [30].

Aspect	Retained foundation	Addition in this article
Benchmark and task	Historical Wikidata correction corpus, local direct-statement repair task, and six-slot add/delete action.	A 50% stratified benchmark with locally attached primary and secondary constraints.
Graph and model	Violation-centred local context, GINE encoding, graph pooling, and six-slot decoder.	Factor nodes with typed scopes, symbolic validators, satisfaction heads, and pressure channels. The Direct–Passive GNN reuses the passive design but is retrained for this study.
Learning	Historical slot-wise supervision.	Factor supervision, candidate-level safety objectives, and a satisfaction-only reranker.
Evaluation	Exact historical-edit matching and primary-constraint repair checks.	Reconstructed post-edit states; local, secondary-transition, disruption, and base-statement-preservation metrics; factor probes, pressure masking, and candidate-oracle analysis.

- 53 **1. Predictive fidelity.** How accurately does the factor-based system reproduce historical repairs
54 relative to the available neural and heuristic systems?
- 55 **2. Mechanistic inspectability.** Do executable constraint factors encode signals aligned with
56 their symbolic satisfaction labels, and do their message channels materially influence repair
57 predictions?
- 58 **3. Metric agreement.** Does greater historical fidelity correspond to better post-edit constraint
59 behaviour, or do these evaluation dimensions diverge?
- 60 **4. Decision-level control.** Can changes to the training objective or candidate-selection strategy
61 improve secondary-constraint behaviour without substantially sacrificing historical fidelity?
- 62 **5. Metric vulnerability.** Can local satisfaction assign high scores to repairs that delete the
63 statement witnessing the violation rather than preserving its evidence?

64 Contributions.

65 This paper makes four contributions. First, it introduces executable constraint factors, a rep-
66 resentation that gives supported locally applicable Wikidata constraints typed scopes, symbolic
67 validators, neural satisfaction heads, and explicit message channels within a violation-centred
68 repair graph. Second, it develops an evaluation framework that separates historical fidelity, post-
69 edit rule behaviour, secondary improvements and regressions, disruption, and base-statement
70 preservation. Third, it provides mechanistic evidence that the factor pathway encodes signals
71 aligned with its symbolic satisfaction labels and materially influences repair prediction. Fourth,
72 it characterises satisfaction-by-deletion with a deterministic counterexample, showing why rule
73 satisfaction alone is insufficient as either a repair objective or an evaluation criterion.

74 Table 1 distinguishes the foundations retained from the related conference paper from the
75 additions evaluated here. All numerical results in this article come from the present experiments
76 rather than being carried over from the conference paper.

77 **2 Related Work**

78 **2.1 Wikidata, constraints, and validation**

79 Wikidata is a collaborative, evolving knowledge base featuring a data model built on typed
80 statements, qualifiers, references, and community-driven curation [31, 5, 23, 24]. Crucially, its
81 property constraints function as guiding principles that flag violations rather than rigid database
82 constraints that block edits [33, 18, 32].

83 This distinction fundamentally shapes repair learning. When the materialised local graph
84 contains the evidence required by a symbolic validator, the validator can determine whether its
85 local condition is satisfied. Historical edit logs, however, record curator choices that need not
86 coincide with locally satisfying or evidence-preserving repairs. Learning directly from past edits
87 therefore optimises historical fidelity to observed curation, which may diverge from post-edit rule
88 behaviour.

89 For standard RDF graphs, languages like SHACL and ShEx provide structural validation
90 frameworks [14, 26]. However, Wikidata’s Wikibase data model and property-constraint eco-
91 system require more specialised formalisations. Recent work has formalised Wikidata property
92 constraints using SHACL and SPARQL [6], MARS-style logical frameworks [16, 17], and dedic-
93 ated Wikibase-oriented shape languages such as WShEx [15].

94 **2.2 Symbolic constraint violation repair**

95 Symbolic repair studies how violations can be explained and corrected under formal semantics.
96 Database repair and consistent query answering provide a long-running foundation for minimal-
97 ity and repair semantics [4]. SHACL work studies explanations and preferred minimal repairs,
98 including answer-set-programming approaches [1, 2, 3], while axiom weakening seeks to preserve
99 content [29]. These symbolic approaches define validity, minimality, and preservation criteria but
100 do not select among soft-constraint repairs using incomplete local evidence and curator prefer-
101 ences. This motivates learning-based repair models that use symbolic validation as supervision,
102 structure, and evaluation while assessing historical fidelity and post-edit behaviour within the
103 available local evidence separately.

104 Ferranti et al. [7] formalise Wikidata repair patterns, distinguishing A-box edits to statements
105 and qualifiers from T-box changes to constraint definitions and class hierarchies. Their taxonomy
106 characterises a broad symbolic repair space and the repair patterns observed in curator edits; we
107 learn direct-statement A-box repairs and evaluate them with local validators.

108 **2.3 Learning-based KG repair and correction**

109 Historical edits have trained statistical and neural correction systems [21, 22]. Related KG quality
110 work includes link prediction, anomaly detection, plausibility scoring, and refinement [19, 28, 38],
111 but repair requires a concrete edit whose post-edit effects can be checked. Vázquez et al. [30]
112 encode local Wikidata repairs with multi-relational GNNs. We retain that prediction task while
113 making applicable constraints explicit and separating historical fidelity from symbolic outcomes
114 and base-statement preservation.

115 **2.4 Neuro-symbolic constraints and factor graphs**

116 A large neuro-symbolic literature integrates symbolic knowledge into neural learning through dif-
117 ferentiable or logical losses [35, 8, 37], constrained structured prediction [13], factor-graph neural

message passing [39, 9], and constrained decoding [10]. Our work is closest in spirit to factor-mediated message passing: constraints are represented as local objects that communicate with variables. The difference is that our factors are grounded Wikidata constraint instances associated with symbolic validators that produce state-specific satisfaction labels, rather than generic learned compatibility functions. We do not assume that soft constraint integration guarantees a valid or useful output. Instead, we use symbolic validation after applying predicted edits to examine divergence among local satisfaction objectives, historical fidelity, post-edit rule behaviour, and base-statement preservation.

3 Background: Wikidata Constraints and Repairs

3.1 Statements, constraints, violations, and repairs

Wikidata represents knowledge as statements $p(s, o)$, where s is an item, p is a property, and o is an item or literal [34, 31]. We study additions and deletions of complete direct statements. A property constraint is metadata attached to a property. It states how that property is normally expected to be used. For example, **single** limits cardinality, **valueType** restricts value classes, **inverse** requires a corresponding statement, and **conflictWith** forbids a co-occurring pattern.

We call the statement that triggers the violation the *base statement*. Some violations also depend on local evidence: **conflictWith** may require an incompatible context statement, while **distinct** or **single** may require another statement sharing a value or property. An **inverse** or **itemRequiresStatement** violation may instead involve missing expected evidence. We call the symbolic pattern of present or missing statements and constraint metadata the *witness pattern*. A *repair action* is an edit proposed to address it; success is determined by post-edit validation.

For a constraint occurrence c and a bounded symbolic state G , the local validation procedure returns *satisfied*, *violated*, or *unknown*. It returns *unknown* when no supported family-specific validator applies or when the required constraint parameters or local evidence are unavailable. We call c *checkable* on G in the first two cases and *uncheckable* in the third. Thus, uncheckability denotes insufficient support for a Boolean decision rather than a violation. Checkability is state-dependent and may change after an edit.

Although Wikidata itself does not enforce a strict description-logic separation, the A-box/T-box abstraction is useful for repair analysis. Ferranti et al. [7] distinguish A-box repairs, which modify base statements, context statements, or qualifiers, from T-box repairs, which modify constraint definitions or hierarchy information. This distinction defines the scope of our work. We study direct-statement A-box repair proposals learned from historical edits.

Concrete constraint definitions and locally materialised type- or subclass-style facts are used to instantiate constraints on the input subgraph and evaluate satisfaction, but the systems evaluated in this paper do not propose edits to constraint definitions or class hierarchies. Likewise, the full Wikidata repair space includes qualifiers, ranks, and references; these matter for complete Wikidata repair, but they are outside the six-slot direct-statement prediction task evaluated here.

4 Task and Problem Formulation

4.1 Repair instances and actions

A repair instance is a constraint violation paired with its local context and historical edit:

$$i = (S_i, \mathcal{W}_i, \mathcal{A}_i, \mathcal{C}_i, \Delta_i^{\text{hist}})$$

It consists of the base statement $S_i = (s_i, p_i, o_i)$, additional witness statements $\mathcal{W}_i$, the one-hop outgoing A-box neighbourhood $\mathcal{A}_i$, locally attached constraints $\mathcal{C}_i$, and the historical repair Δ_i^{hist} .

The attached set includes the primary constraint c_i^* , i.e. the constraint associated with the historical violation record, and constraints on predicates in the base, witness, and materialised entity neighbourhoods. A constraint definition contains its constrained property and parameters such as required properties, allowed classes or values, inverses, or conflicts. All claims are restricted to this bounded evidence: type validation uses locally present P31 (*instance of*) and P279 (*subclass of*) facts; no transitive class hierarchy is materialised. Only families with implemented validators, $\mathcal{C}_i^{\text{fac}} \subseteq \mathcal{C}_i$, execute as factors.

An edit consists of at most one complete statement addition and deletion:

$$\Delta = ((s_i^+, p_i^+, o_i^+), (s_i^-, p_i^-, o_i^-)).$$

Either operation may be absent; a replacement pairs a deletion with an addition. Both the historical edit Δ_i^{hist} and the predicted edit Δ_i^{pred} share this representation.

4.2 Graph materialisation and repair states

The stored factorised input graph for instance i contains local A-box entity and predicate-occurrence nodes $\mathcal{V}_i$, factor nodes $\mathcal{F}_i = \{f_c : c \in \mathcal{C}_i^{\text{fac}}\}$, and branches encoding each factor’s constraint parameters. For each locally materialised statement occurrence j , with triple (s_j, p_j, o_j) , the stored graph contains the directed path $s_j \rightarrow v_{p_j, j} \rightarrow o_j$, where $v_{p_j, j}$ is a fresh predicate-occurrence node. Thus, repeated uses of the same property have distinct local predicate nodes. Each factor has bidirectional, role-typed scope edges to selected local subject, predicate, and object nodes; its definition branches remain directed.

Figure 1 illustrates the factorised representation and two possible local edits.

This stored neural input graph is distinct from the symbolic evidence state used for evaluation. From the same instance record, the evaluator reconstructs G_i^{pre} as a bounded set of local A-box statements. The historical and predicted edits are applied to this symbolic state to obtain

$$G_i^{\text{hist}} = \text{apply}(G_i^{\text{pre}}, \Delta_i^{\text{hist}}), \quad G_i^{\text{pred}} = \text{apply}(G_i^{\text{pre}}, \Delta_i^{\text{pred}}).$$

Because constraint symbolic evaluation is restricted to the reconstructed local graph, an operation is applied only when its subject is represented and its predicate occurs within the instance’s local predicate closure. Otherwise, the available evidence is insufficient to evaluate its local consequences: the operation remains part of the predicted repair for historical-fidelity and disruption metrics, but it does not modify the symbolic state used to compute constraint-satisfaction metrics.

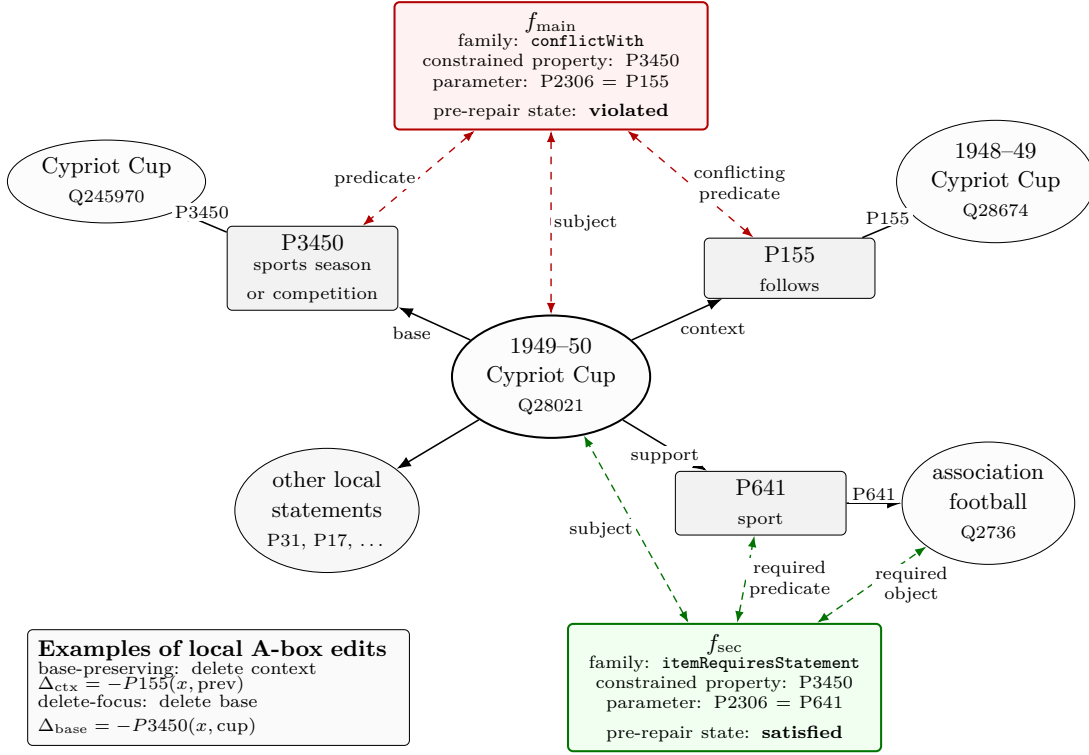
4.3 Local validator semantics

Table 2 specifies the exact bounded semantics used by the evaluator for every supported constraint family. These validators are role-anchored approximations of the corresponding Wikidata constraints: they operate on the reconstructed local state of one repair instance, rather than on the complete Wikidata graph.

For a constraint occurrence c , let p_c denote its constrained property, and let $\mathcal{P}_i$ be the predicate closure represented for instance i . For a reconstructed state G , define

$$p^G(s) = \{o \mid p(s, o) \in G\},$$

the set of locally represented values of property p for entity s . We call an entity–property pair (s, p) *locally decidable* in G , written $\text{dec}_G(s, p)$, when s is represented, $p \in \mathcal{P}_i$, and no operation that could not be applied to the local state targets that pair. On a locally decidable pair, the evaluator adopts a bounded closed-world interpretation: if $p^G(s) = \emptyset$, the property is treated as



■ **Figure 1** Example violation-centred subgraph with executable constraint factors. Circles denote item/value variables, rounded boxes denote predicate occurrences, and colored boxes denote property constraints; solid arrows are local A-box statement paths, while dashed arrows are bidirectional factor-scope links.

locally absent. This is an assumption about the materialised instance, not about completeness of Wikidata.

An operation whose subject is not represented or whose predicate lies outside $\mathcal{P}_i$ is not applied to G . A validator that subsequently requires the affected entity–property pair therefore returns *unknown*. The type-related validators are deliberately more conservative: if their trigger remains present but no local typing statement is available, they also return *unknown*, rather than treating the missing type evidence as a violation.

After identifier resolution, the evaluator constructs the following definition-parameter sets:

- Req_c , the required properties given by P2306 (*property*);
- Allow_c , the allowed items given by P2305 (*item of property constraint*);
- Class_c , the allowed classes given by P2308 (*class*);
- Rel_c , the typing relations given by P2309 (*relation*), defaulting to P31 (*instance of*) and P279 (*subclass of*);
- Inv_c , the inverse properties given by P1696 (*inverse property*), defaulting to p_c .

For **conflictWith**, Conf_c contains the resolved property-valued parameters, with the recorded witness predicate used as a fallback when this set is empty. Only successfully resolved identifiers enter these sets. Whenever a condition below requires a non-empty parameter set, an empty set makes the occurrence uncheckable.

The *anchor* column identifies the fixed repair-instance role from which each validator begins its local queries. Validators may inspect statements about the base subject s_i , the base value o_i ,

222 or the recorded witness subject. Supported primary and secondary constraint occurrences are
 223 evaluated against these shared anchors; they are not independently re-grounded over every poten-
 224 tially matching statement in the neighbourhood. Consequently, several validators intentionally
 225 inspect only the focal base value or the recorded witness.

■ **Table 2** Family-specific semantics of the bounded local validators. The satisfaction condition is evaluated only after the checkability condition holds.

Family	Anchor and parameters	Checkable when	Satisfied when	Bounded-evidence limitation
conflict With	Subject s_i ; $p_c, \text{Conf}_c, \text{Allow}_c$	$\text{Conf}_c \neq \emptyset$, $\text{dec}_G(s_i, p_c)$, and $\text{dec}_G(s_i, p)$ for every $p \in \text{Conf}_c$.	$p_c^G(s_i) = \emptyset$, or, for every $p \in \text{Conf}_c$, either $\text{Allow}_c = \emptyset$ and $p^G(s_i) = \emptyset$, or $\text{Allow}_c \neq \emptyset$ and $p^G(s_i) \cap \text{Allow}_c = \emptyset$.	–
distinct	Base S_i ; recorded witness (s_w, p_w, o_w)	$\text{dec}_G(s_i, p_c)$; if $S_i \in G$, the recorded witness is complete and $p_w = p_c$.	It is not the case that $S_i \in G$, $p_w = p_c$, $o_w = o_i$, and $s_w \neq s_i$.	Compares only locally represented statements.
inverse	Base S_i ; p_c, Inv_c	$p_i = p_c$ and $\text{dec}_G(s_i, p_c)$; if $S_i \in G$, then $\text{Inv}_c \neq \emptyset$ and $\text{dec}_G(o_i, p)$ for every $p \in \text{Inv}_c$.	$S_i \notin G$, or $p(o_i, s_i) \in G$ for some $p \in \text{Inv}_c$.	Compares only locally represented statements.
symmetric	Base S_i ; p_c	As for inverse , with the reverse property fixed to p_c .	$S_i \notin G$, or $p_c(o_i, s_i) \in G$.	–
item Requires Statement	Subject s_i ; $p_c, \text{Req}_c, \text{Allow}_c$	$\text{Req}_c \neq \emptyset$ and $\text{dec}_G(s_i, p_c)$; if $p_c^G(s_i) \neq \emptyset$, then $\text{dec}_G(s_i, p)$ for every $p \in \text{Req}_c$.	$p_c^G(s_i) = \emptyset$, or, for every $p \in \text{Req}_c$, either $\text{Allow}_c = \emptyset$ and $p^G(s_i) \neq \emptyset$, or $\text{Allow}_c \neq \emptyset$ and $p^G(s_i) \cap \text{Allow}_c \neq \emptyset$.	–

■ **Table 2** Family-specific semantics of the bounded local validators (continued).

Family	Anchor and parameters	Checkable when	Satisfied when	Bounded-evidence limitation
value Requires Statement	Base value o_i ; $p_c, \text{Req}_c, \text{Allow}_c$	$p_i = p_c$ and $\text{dec}_G(s_i, p_c)$; if $S_i \in G$, then $\text{Req}_c \neq \emptyset$ and $\text{dec}_G(o_i, p)$ for every $p \in \text{Req}_c$.	$S_i \notin G$, or, for every $p \in \text{Req}_c$, either $\text{Allow}_c = \emptyset$ and $p^G(o_i) \neq \emptyset$, or $\text{Allow}_c \neq \emptyset$ and $p_c^G(s_i) \subseteq \text{Allow}_c$.	Only the base value p_i can be inspected
oneOf	Subject s_i ; p_c, Allow_c	$\text{Allow}_c \neq \emptyset$ and $\text{dec}_G(s_i, p_c)$.	$ p_c^G(s_i) \leq 1$.	–
single type	Subject s_i ; p_c Subject s_i ; $p_c, \text{Class}_c, \text{Rel}_c$	$\text{dec}_G(s_i, p_c)$. $\text{Class}_c \neq \emptyset$ and $\text{dec}_G(s_i, p_c)$; if $p_c^G(s_i) \neq \emptyset$, then $\text{Rel}_c \neq \emptyset$, $\text{dec}_G(s_i, p)$ for every $p \in \text{Rel}_c$, and $\bigcup_{p \in \text{Rel}_c} p^G(s_i) \neq \emptyset$.	$p_c^G(s_i) = \emptyset$, or $\left(\bigcup_{p \in \text{Rel}_c} p^G(s_i) \right) \cap \text{Class}_c \neq \emptyset$.	– Uses direct local class matches only; it does not follow a global transitive class hierarchy.
valueType	Focal value o_i ; $p_c, \text{Class}_c, \text{Rel}_c$	o_i has a resolved identifier, $\text{Class}_c \neq \emptyset$, and $\text{dec}_G(s_i, p_c)$; if $p_c^G(s_i) \neq \emptyset$, then $\text{Rel}_c \neq \emptyset$, $\text{dec}_G(o_i, p)$ for every $p \in \text{Rel}_c$, and $\bigcup_{p \in \text{Rel}_c} p^G(o_i) \neq \emptyset$.	$p_c^G(s_i) = \emptyset$, or $\left(\bigcup_{p \in \text{Rel}_c} p^G(o_i) \right) \cap \text{Class}_c \neq \emptyset$.	No global transitive hierarchy is followed.

These rules also determine how deletion affects validation. The validators for **inverse**, **symmetric**, **valueRequiresStatement**, **oneOf**, and **distinct** use the exact base statement as their trigger. Once $S_i \notin G$, they are satisfied whenever the remaining checkability conditions hold.

The other families evaluate property sets on fixed local anchors, so deleting S_i has no uniform effect. Depending on the role of p_i in a particular constraint occurrence, it may remove the constrained-property trigger, conflicting evidence, a required statement, a typing statement, or merely one tested value, as for **oneOf**. For **single**, reducing $|p_c^G(s_i)|$ to at most one is sufficient. For **type** and **valueType**, removing the last locally represented typing statement while leaving the trigger active makes the occurrence unknown rather than violated. The **valueType** validator is additionally role-anchored: another p_c -value can keep the trigger active even though the class test remains attached to the recorded o_i . An operation outside the represented subject or predicate scope leaves G unchanged, and any validator that requires the corresponding unresolved pair becomes uncheckable.

4.4 Evaluation views

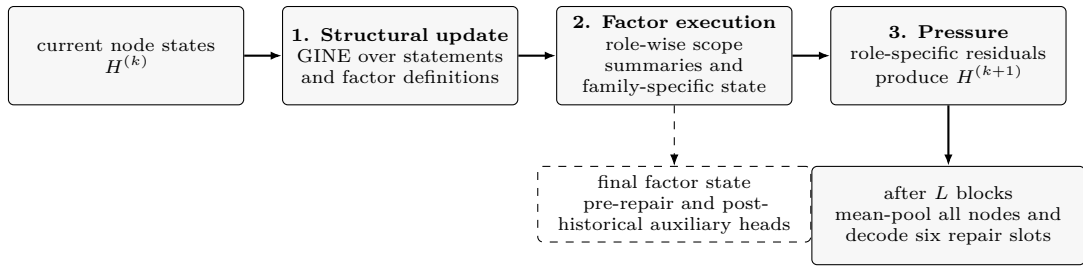
Given the stored representation of $(G_i^{\text{pre}}, \mathcal{C}_i)$, the system predicts Δ_i^{pred} . *Historical fidelity* compares that action with Δ_i^{hist} ; *symbolic repair quality* rechecks $\mathcal{C}_i$ on G_i^{pred} . They are deliberately separate because imitation may introduce new violations while satisfaction may increase through base statement deletion.

5 Executable Constraint Factors

The Direct-Factor GNN augments a relational graph encoder with one neural computational unit for each locally attached, validator-supported constraint occurrence. The central idea is to

separate two kinds of computation. Ordinary graph message passing represents the statements and constraint definitions in the instance. A *factor step* then reads the current representations of the nodes that play the predicate, subject, and object roles of a particular constraint, constructs a constraint-specific state, and sends a learned residual back to those nodes. Repeating this read–compute–write cycle allows constraint context to influence the eventual repair prediction.

At a high level, a message-passing GNN associates every graph node with a learned numerical summary and every layer updates that summary from neighbouring nodes. No individual coordinate needs a predefined interpretation. GINE—a Graph Isomorphism Network that also uses edge features—is the structural update used here, so the relation connecting two nodes can affect the message [36, 12]. Our architecture inserts factor execution after each such update, as summarised in Figure 2. The symbolic validators are not called during this neural forward pass; they provide supervision and are used separately for post-edit evaluation.



■ **Figure 2** Data flow in the executable-factor architecture. Structural GINE edges and factor-scope links have different roles: GINE encodes statements and factor definitions, whereas the factor step reads and writes through typed predicate/subject/object scope links. The post-historical head is used only for auxiliary training and validation.

Running example.

For the `conflictWith` case in Figure 1, GINE first contextualises the two local statements and the factor-definition branch. The factor then reads the predicate occurrences and the subject/object nodes connected to its typed scope, constructs a family-specific state, and returns learned pressure to those nodes. It is not given a hand-written choice between deleting the context statement and deleting the base statement; the graph-level decoder learns that choice from historical actions.

5.1 Step 1: structural encoding of the repair instance

Let $\mathcal{Z}_i$ denote the complete node set of the stored factorised input graph introduced in Section 4.2. In addition to $\mathcal{V}_i$ and $\mathcal{F}_i$, it includes the predicate-occurrence and object nodes on the factor-definition branches. The stored graph retains the expanded statement paths introduced there. A constraint-parameter pair (q, a) is represented analogously as $f_c \rightarrow v_q^{\text{par}} \rightarrow a$, with a fresh parameter-predicate occurrence node for each pair.

The Direct–Factor GNN uses node-ID encoding. Each node receives a learned lookup vector for its identifier and a learned role-code embedding. The role code marks whether the node is the subject, predicate occurrence, or object of the base statement S_i ; nodes with none of these roles receive the neutral code. An initialisation MLP maps their concatenation to the model state:

$$h_z^{(0)} = \text{MLP}_{\text{init}}(E_{\text{node}}(\text{id}(z)) \parallel E_{\text{role}}(\rho_z)).$$

Fresh occurrences of the same property occupy different graph-node positions but use the same property identifier at initialisation.

For structural GINE message passing, each statement occurrence j contributes a directed $s_j \rightarrow o_j$ edge whose edge attribute points to $v_{p_j,j}$; a factor-definition pair similarly contributes $f_c \rightarrow a$, with v_q^{par} as its edge attribute. Let $\text{predocc}(u, z)$ denote the predicate-occurrence node associated with the structural edge $u \rightarrow z$. At block k , an edge MLP derives the edge feature from the current state of that predicate occurrence:

$$e_{u \rightarrow z}^{(k)} = \text{MLP}_{\text{edge}} \left(h_{\text{predocc}(u,z)}^{(k)} \right).$$

GINE then produces the intermediate structural state

$$\tilde{h}_z^{(k+1)} = \text{GINE}_k \left(h_z^{(k)}, \left\{ \left(h_u^{(k)}, e_{u \rightarrow z}^{(k)} \right) : u \in \mathcal{N}_{\text{str}}^-(z) \right\} \right),$$

where $\mathcal{N}_{\text{str}}^-(z)$ contains the incoming statement and factor-definition neighbours of z . A-box nodes within a factor scope subsequently receive constraint pressure; all other nodes retain $h_z^{(k+1)} = \tilde{h}_z^{(k+1)}$ as input for the next neural layer. The Direct-Factor GNN applies three such blocks, with each structural update followed by factor execution and pressure.

5.2 Step 2: constraint-factor read and write

Each factor is a grounded occurrence with a concrete definition, a family $t(c)$, such as **single** or **conflictWith**, and a typed local scope. For $r \in \{\text{pred}, \text{subj}, \text{obj}\}$, define

$$\mathcal{R}_{c,r} = \{v \in \mathcal{Z}_i : f_c \text{ is linked to } v \text{ with role } r\}, \quad n_{c,r} = |\mathcal{R}_{c,r}|.$$

A role may contain no node, one node, or several matching occurrences. After the structural update, the executor forms a separate mean for every role:

$$\bar{h}_{c,r}^{(k+1)} = \frac{1}{\max(1, n_{c,r})} \sum_{v \in \mathcal{R}_{c,r}} \tilde{h}_v^{(k+1)}.$$

The summary is the zero vector when the role is empty. The executor input concatenates the structurally updated factor-node state, the three role summaries, and the logarithm of each role count:

$$\phi_c^{(k+1)} = \left[\tilde{h}_{f_c}^{(k+1)} \parallel \bar{h}_{c,\text{pred}}^{(k+1)} \parallel \bar{h}_{c,\text{subj}}^{(k+1)} \parallel \bar{h}_{c,\text{obj}}^{(k+1)} \parallel \log(1 + n_{c,\text{pred}}) \parallel \log(1 + n_{c,\text{subj}}) \parallel \log(1 + n_{c,\text{obj}}) \right].$$

The count features distinguish an empty role from one matched by one or several local statements.

All factors belonging to the same constraint family share a MLP $F_{t(c)}$. Because constraint families express different satisfaction conditions, this parameterisation lets the factor readout specialise to each family’s semantics while sharing statistical strength across all grounded occurrences of that family:

$$(a_c^{(k+1)}, \ell_{c,\text{pre}}^{(k+1)}) = F_{t(c)} \left(\phi_c^{(k+1)} \right).$$

The latent state $a_c^{(k+1)}$ is the representation used for communication. The accompanying raw score (logit), whose sigmoid estimates satisfaction, is not used to gate pressure. After the final residual update, the same executor is applied once more to the completed node states to obtain a final factor readout a_c^{out} and the pre-repair logit $\ell_{c,\text{pre}}$ used by the auxiliary loss described below.

For every factor-node scope link, a family- and role-specific MLP $G_{t(c),r}$ creates a pressure message from the factor state and the recipient’s current state. Conditioning the pressure map on both family and role allows messages to reflect the constraint semantics and the recipient

node’s function within the factor scope, while sharing parameters across all links with the same family–role combination:

$$m_{c \rightarrow v, r}^{(k+1)} = G_{t(c), r} \left(a_c^{(k+1)} \parallel \tilde{h}_v^{(k+1)} \right).$$

Messages arriving at the same node are averaged and added as a residual,

$$h_v^{(k+1)} = \tilde{h}_v^{(k+1)} + \lambda \frac{\sum_{(c, r): v \in \mathcal{R}_{c, r}} m_{c \rightarrow v, r}^{(k+1)}}{\max(1, d_v)}, \quad d_v = |\{(c, r) : v \in \mathcal{R}_{c, r}\}|.$$

Nodes without an incoming factor–role link remain unchanged. Pressure is therefore a learned, constraint-conditioned residual. It is not a hard rule application: the satisfaction logit is neither thresholded nor used as a symbolic gate, and the neural factor state need not implement the associated validator exactly.

5.3 Step 3: repair prediction and training signals

After the final GNN block, the model mean-pools all node states to obtain a graph-level representation of the repair instance. A shared projection first transforms this vector into a common repair representation. Three role-specific branches then specialise that representation for predicting subjects, predicates, and objects. Each branch feeds two classification heads: one for the addition and one for the deletion. The six heads therefore predict the subject, predicate, and object of the addition and the subject, predicate, and object of the deletion. Subject and object heads range over entity IDs, whereas predicate heads range over property IDs; every vocabulary also contains NONE. The value with the highest score is selected independently for each slot, yielding the six-slot action representation defined in Section 4.1.

The main training signal is the mean cross-entropy against the historical six-slot action; this teaches the six slot classifiers which IDs to select. The Direct–Factor GNN additionally supervises two factor heads. The pre-repair head predicts constraint satisfaction from the final factor readout. The post-historical head also receives the mean learned embedding of the six historical slot IDs and predicts constraint satisfaction after that action. With binary cross-entropy averaged only over factors whose symbolic satisfaction label is defined for the corresponding state, the proposal loss is

$$\mathcal{L}_{\text{factor}}^{(i)} = \frac{1}{6} \sum_q \text{CE}(\ell_{i, q}, y_{i, q}^{\text{hist}}) + 0.1 \overline{\text{BCE}}_{\text{pre}}^{(i)} + 0.1 \overline{\text{BCE}}_{\text{hist}}^{(i)},$$

where an empty eligible factor set contributes zero. The post-historical branch is used only during training and validation, and historical repairs are not supplied at inference. Neither auxiliary logit is fed directly to the edit decoder. The auxiliary losses shape the shared factor executor, while the edit loss also trains the factor–pressure pathway. Reported constraint-satisfaction metrics do not use these logits; they independently revalidate the reconstructed pre- and post-edit states as defined in Section 6.

5.4 Why executable constraint factors?

Representing a constraint definition only as ordinary graph structure leaves generic message passing to discover which local nodes fill semantically different roles and how their evidence should be combined for each constraint family. Executable factors encode this inductive bias directly: each grounded constraint reads its typed predicate, subject, and object scope using

parameters shared by its family, then returns constraint-conditioned residuals to the nodes in that scope. This gives constraint context a direct pathway into the repair representation without prescribing a symbolic repair or invoking the validator during neural inference.

The explicit interface also makes the constraint pathway easier to interrogate. Satisfaction probes test whether factor states encode their symbolic satisfaction labels, while pressure-masking interventions on the trained network test whether messages from primary and non-primary factors affect the predicted repair. These diagnostics distinguish label alignment from pathway use; neither establishes validator-equivalent reasoning or safe repair. Those outcomes require the post-edit symbolic evaluation defined in Section 6.

6 Evaluation Framework

Historical fidelity compares Δ_i^{pred} with Δ_i^{hist} : **precision, recall, and Micro-F1** pool matches between their additions and deletions. Constraint-satisfaction metrics instead compare the reconstructed states G_i^{pre} and G_i^{pred} .

Constraint support.

The family-specific validators in Table 2 produce the outcomes used here. Let $\chi_c(G) \in \{0, 1\}$ indicate whether c is checkable on G . When it is checkable, let $s_c(G) \in \{0, 1\}$ indicate whether the constraint is satisfied. Define the post-edit and common supports as

$$\mathcal{C}_i^{\text{pred}} = \{c \in \mathcal{C}_i : \chi_c(G_i^{\text{pred}}) = 1\}, \quad \mathcal{C}_i^{\leftrightarrow} = \{c \in \mathcal{C}_i : \chi_c(G_i^{\text{pre}}) = 1 \wedge \chi_c(G_i^{\text{pred}}) = 1\}.$$

We call $\mathcal{C}_i^{\leftrightarrow}$ the *common support*: it contains exactly those constraint occurrences that are checkable both before and after the predicted repair, so their satisfaction values can be compared. Every constraint-satisfaction metric reports its eligible denominator; an uncheckable constraint contributes neither a positive result nor an artificial zero.

Primary and local satisfaction.

Let the Primary-Fix eligibility set be the set of instances whose primary constraint is checkable and violated before repair.

$$\mathcal{I}_{\text{PF}} = \{i : \chi_{c_i^*}(G_i^{\text{pre}}) = 1 \wedge s_{c_i^*}(G_i^{\text{pre}}) = 0\}.$$

The **Primary Fix Rate (PFR)** measures the proportion of eligible instances whose primary constraint is fixed by the system’s prediction.

$$\text{PFR} = \frac{\sum_{i \in \mathcal{I}_{\text{PF}}} \mathbf{1}[\chi_{c_i^*}(G_i^{\text{pred}}) = 1 \wedge s_{c_i^*}(G_i^{\text{pred}}) = 1]}{|\mathcal{I}_{\text{PF}}|}.$$

The denominator is fixed by the pre-edit state: an eligible primary that becomes uncheckable after the predicted edit counts as a failure rather than leaving the support. **Post-edit Local Satisfaction** $\text{LocalSat}^{\text{pred}}$ pools satisfied constraints, while the **Change in Local Satisfaction** $\Delta\text{LocalSat}$ uses common support so altered coverage is not mistaken for improvement:

$$\text{LocalSat}^{\text{pred}} = \frac{\sum_i \sum_{c \in \mathcal{C}_i^{\text{pred}}} s_c(G_i^{\text{pred}})}{\sum_i |\mathcal{C}_i^{\text{pred}}|}, \quad \Delta\text{LocalSat} = \frac{\sum_i \sum_{c \in \mathcal{C}_i^{\leftrightarrow}} [s_c(G_i^{\text{pred}}) - s_c(G_i^{\text{pre}})]}{\sum_i |\mathcal{C}_i^{\leftrightarrow}|}.$$

387 **Secondary satisfaction.**

388 For secondary satisfaction, exclude the primary constraint from the common support $\mathcal{C}_i^{\text{sec}} =$
 389 $\mathcal{C}_i^{\leftrightarrow} \setminus \{c_i^*\}$. The occurrence-pooled **Secondary Improvement Rate (SIR)** measures how often
 390 a secondary constraint violated before repair becomes satisfied; the **Secondary Regression**
 391 **Rate (SRR)** measures the converse transition from satisfied to violated:

$$\begin{aligned} \text{SIR} &= \frac{\sum_i \sum_{c \in \mathcal{C}_i^{\text{sec}}} \mathbf{1}[s_c(G_i^{\text{pre}}) = 0 \wedge s_c(G_i^{\text{pred}}) = 1]}{\sum_i \sum_{c \in \mathcal{C}_i^{\text{sec}}} \mathbf{1}[s_c(G_i^{\text{pre}}) = 0]}, \\ \text{SRR} &= \frac{\sum_i \sum_{c \in \mathcal{C}_i^{\text{sec}}} \mathbf{1}[s_c(G_i^{\text{pre}}) = 1 \wedge s_c(G_i^{\text{pred}}) = 0]}{\sum_i \sum_{c \in \mathcal{C}_i^{\text{sec}}} \mathbf{1}[s_c(G_i^{\text{pre}}) = 1]}. \end{aligned}$$

393 An instance contributes nothing to a rate when it has no secondary occurrence eligible for that
 394 rate’s denominator. Higher SIR and lower SRR are preferred.

395 **Disruption and base-statement preservation.**

396 Constraint satisfaction alone does not reveal how a repair achieved its score: a targeted correction,
 397 a multi-operation rewrite, and removal of the statement witnessing the violation may all improve
 398 local validation outcomes. We therefore complement the satisfaction metrics with diagnostics of
 399 operation count and base-statement preservation.

400 **Disruption** is the number of complete predicted operation groups. A partially populated slot
 401 group is not an operation and contributes zero. Because the task permits at most one addition
 402 and one deletion, per-instance disruption belongs to $\{0, 1, 2\}$: the values denote no operation, one
 403 addition or deletion, and both operations, respectively. We report its mean together with the
 404 separate **mean addition and deletion counts**.

405 The base statement S_i is guaranteed to belong to G_i^{pre} . Let $\text{BaseRemoved}_i = \mathbf{1}[S_i \notin G_i^{\text{pred}}]$.
 406 **Base-deletion Rate** averages this indicator, measuring whether the base is absent from the
 407 final reconstructed state. **Deletes-base-action Rate** instead measures how often the resolved
 408 deletion equals S_i . A delete-and-reinsert repair therefore counts as deleting the base action but
 409 not as removing the base from G_i^{pred} .

410 The **Evidence-Preserving Primary Fix rate (EPPF)** credits an eligible instance only
 411 when the primary is fixed and the base remains present:

$$\text{EPPF} = \frac{\sum_{i \in \mathcal{I}_{\text{PF}}} \mathbf{1}[\chi_{c_i^*}(G_i^{\text{pred}}) = 1 \wedge s_{c_i^*}(G_i^{\text{pred}}) = 1 \wedge S_i \in G_i^{\text{pred}}]}{|\mathcal{I}_{\text{PF}}|}.$$

413 **Vacuous Improvement** is the fraction of all test instances for which $\text{BaseRemoved}_i = 1$ and
 414 $\sum_{c \in \mathcal{C}_i^{\leftrightarrow}} [s_c(G_i^{\text{pred}}) - s_c(G_i^{\text{pre}})] > 0$. An empty common support has sum zero and therefore does
 415 not count. Here, *base-statement preservation* means retention of S_i , not preservation of qualifiers,
 416 references, or other context. These diagnostics distinguish deletion-based improvements but do
 417 not declare every deletion incorrect.

418 **7 Experimental Setup**419 **7.1 Benchmark and constraint coverage**

420 The benchmark derives from the cleaned and annotated Wikidata constraint-correction corpus of
 421 Pellissier Tanon and colleagues [21, 22, 20], extracted from the 1 July 2018 Wikidata full-history
 422 dump. Each source row identifies one extracted correction: its constraint definition, the revision

■ **Table 3** Per-instance descriptive statistics for the 143,316 held-out test cases. A-box counts use distinct statements in the reconstructed pre-edit state; graph counts refer to the stored factorised neural input.

Quantity	Mean	Median	95th pct.	Max.
Local A-box statements	26.55	17	74	558
Locally attached constraints	101.05	83	266	1,092
Executable factors	60.60	51	168	662
Factorised-graph nodes	1,054.93	765	4,337	8,398
Factorised-graph directed edges	1,811.86	1,444	5,010	66,894

that resolved the violation, one or two violation statements, one or two edit operations, and local entity descriptions. We treat each retained correction row as one repair instance of the form defined in Section 4.1. The preprocessing does not reconstruct or decompose complete revisions, and several rows may refer to the same revision. All 1,910,794 source rows used here have a matching constraint definition; no additional row-level deduplication is applied. Their historical actions contain either one operation or one addition and one deletion, and therefore map without truncation to the six-slot action in Section 4.1.

We discard the supplied partitions and repartition correction rows 70/15/15, stratified by primary constraint family with seed 42, obtaining 1,337,555 training, 286,619 validation, and 286,620 test instances. Identifier frequencies are then computed on this training partition: identifiers occurring fewer than 100 times are mapped to UNK in every split, while reserved placeholders and constraint-definition identifiers are retained. Thus, the minimum-occurrence threshold changes the vocabulary rather than removing repair instances. The paper benchmark retains 50% of every non-empty stratum defined by split, primary family, and number of locally attached constraints (bins 1–32, 33–64, 65–83, 84–107, 108, 109–160, 161–267, and 268+), using seed 42 and retaining at least one row per stratum. This yields 955,405 instances: 668,779 train, 143,310 validation, and 143,316 test. The split unit is a correction row rather than an entity or constraint definition, so both may recur across partitions; Section 9 discusses the resulting in-distribution scope.

Each instance designates the constraint named by its historical violation record as primary; all other locally attached constraint occurrences are secondary. Primary factors span `conflictWith`, `distinct`, `inverse`, `itemRequiresStatement`, `oneOf`, `single`, `type`, `valueRequiresStatement`, and `valueType`; `symmetric` appears only among secondary factors. A constraint family is supported when both its local validator, with the semantics in Table 2, and its factor construction are defined. Supported occurrences become executable factors and receive symbolic satisfaction labels; unsupported occurrences remain descriptive metadata and contribute no Boolean satisfaction outcomes.

The factor-supervision labels cover 58,049,928 supported constraint occurrences. Of these occurrences, 64.9% are checkable in the pre-repair state and 67.9% after applying the historical repair. These labels provide training targets for the auxiliary factor heads.

7.2 Proposal architectures and baselines

The Direct group contains two neural systems whose six output slots are decoded independently. The Direct-Factor GNN is the executable-factor architecture described in Section 5; it uses family-specific executors, pressure, and auxiliary factor heads. The Direct-Passive GNN is the neural baseline and uses a passive GINE encoder. Its graph retains the primary constraint-definition branch but has no executable factor scopes, auxiliary factor heads, or pressure.

The Baseline group contains four non-neural comparators. Baseline–DB (Delete Base) and Baseline–AM (Add Mirror) are deterministic symbolic heuristics. Baseline–DB always deletes S_i . For inverse and symmetric constraints, Baseline–AM reverses the base statement, using a known inverse property when available and otherwise retaining the original property; it predicts no edit for other families. Baseline–FM (Family Majority) and Baseline–DM (Definition Majority) are statistical: each predicts the most frequent training target independently for every action slot, conditioned on the constraint family or concrete definition, respectively. Because concrete definitions may occur across the random split, Baseline–DM measures in-distribution memorisation, never test-label access.

7.3 Decision-level candidate systems

The Direct–Factor GNN gives constraint information a pathway into the repair representation, but its six heads choose slot values independently and do not compare complete actions by their constraint consequences. The candidate systems address this limitation by assembling candidate edits and testing whether explicit action selection can improve primary repair and limit secondary regressions while retaining historical fidelity. Candidate–SR provides a narrower diagnostic of what happens when selection optimises local satisfaction alone.

The two Direct systems each decode one six-slot action. Candidate–C (Chooser), Candidate–DP (Direct Penalty), and Candidate–SR (Satisfaction Reranker) instead choose from a candidate set Γ_i derived from proposal logits and constraint heuristics. Let G_{ij} be the state after candidate j , and let a_{ij} be its raw selection score. All three systems normalise over the candidates of that instance as $q_{ij} = \exp(a_{ij}) / \sum_{k=1}^{|\Gamma_i|} \exp(a_{ik})$. Candidate–C and Candidate–SR learn a_{ij} ; for Candidate–DP it is the sum of the six proposal logits selected by the candidate. Let j_i^{hist} denote the historical candidate included when computing the candidate-selection loss during training and validation.

The candidate-level primary term is $s_{ij}^{\text{prim}} = \mathbf{1}[\chi_{c_i^*}(G_{ij}) = 1 \wedge s_{c_i^*}(G_{ij}) = 1]$, so making the primary uncheckable is a failure. Candidate secondary regression is measured against the pre-edit state, not the historical post-edit state. Specifically, let

$$\mathcal{C}_{ij}^{\text{sec}} = \{c \in \mathcal{C}_i \setminus \{c_i^*\} : \chi_c(G_i^{\text{pre}}) = \chi_c(G_{ij}) = 1\}.$$

Then

$$\text{SRR}_{ij}^{\text{cand}} = \frac{\sum_{c \in \mathcal{C}_{ij}^{\text{sec}}} \mathbf{1}[s_c(G_i^{\text{pre}}) = 1 \wedge s_c(G_{ij}) = 0]}{\sum_{c \in \mathcal{C}_{ij}^{\text{sec}}} \mathbf{1}[s_c(G_i^{\text{pre}}) = 1]},$$

with value zero when the denominator is empty. The historical repair enters Candidate–C only as the comparison candidate below; each candidate’s rate uses its own common support.

Candidate–C adds a learned chooser whose goal is to imitate the historical candidate while favouring primary fixes and discouraging secondary regressions beyond those caused by the historical repair. It scores each candidate from the Direct–Factor GNN graph embedding and the mean learned embedding of its six slot IDs. In addition to $\mathcal{L}_{\text{factor}}^{(i)}$, it uses

$$0.25 \left[-\log q_{i, j_i^{\text{hist}}} + 0.25 \sum_j q_{ij} \max\left(0, \text{SRR}_{ij}^{\text{cand}} - \text{SRR}_{i, j_i^{\text{hist}}}^{\text{cand}}\right) + 0.2 \sum_j q_{ij} (1 - s_{ij}^{\text{prim}}) \right].$$

Candidate–DP has no separate chooser: it derives candidate probabilities directly from the proposal slot logits and trains the proposal model itself to favour primary fixes with fewer secondary

regressions. It adds the expected penalty

$$\sum_j q_{ij}(1 - s_{ij}^{\text{prim}}) + 0.5 \sum_j q_{ij} \text{SRR}_{ij}^{\text{cand}}.$$

Candidate-SR is deliberately satisfaction-only: it keeps the Direct-Factor GNN frozen for candidate generation and trains a separate graph encoder and candidate reranker to prefer candidates that satisfy the largest fraction of checkable local constraints. It minimises

$$\mathcal{L}_{\text{SR}}^{(i)} = - \sum_j q_{ij} \bar{s}_{ij},$$

where $\bar{s}_{ij} = |\{c \in \mathcal{C}_i : \chi_c(G_{ij}) = 1 \wedge s_c(G_{ij}) = 1\}| / |\{c \in \mathcal{C}_i : \chi_c(G_{ij}) = 1\}|$, defined as zero when no local constraint is checkable. Because the denominator and candidate SRR support depend on G_{ij} , making a constraint uncheckable can remove it from either quantity; in particular, removing violated constraints can raise $\bar{s}_{ij}$. This vulnerability is the purpose of the Candidate-SR diagnostic. At inference, Candidate-C and Candidate-SR use learned-score argmax, while Candidate-DP uses proposal-logit argmax. These candidate-level terms are instance-normalised training quantities, not the occurrence-pooled metrics in Section 6.

7.4 Optimisation and candidate construction

The Direct-Passive GNN, Direct-Factor GNN, Candidate-C, and Candidate-DP use seed 42, Adam (learning rate 10^{-4} , weight decay 1.1×10^{-4}), batch size 256, gradient clipping 0.5, and at most ten epochs. The Direct-Factor GNN, Candidate-C, and Candidate-DP use 400-dimensional hidden states, three GINE-factor blocks, and dropout 0.17; the Direct-Passive GNN uses 128-dimensional hidden states, one GINE block, and dropout 0.5. Validation loss on validation instances selects checkpoints with patience two. Factorised systems use 128-dimensional node-ID and 16-dimensional role embeddings, GINE predicate-edge attributes, and $\lambda = 0.1$. Active families come only from train/validation, with an error on an unseen test family. Candidate-SR also uses seed 42 and trains a fresh reranker with 64-dimensional slot embeddings, a 128-dimensional candidate hidden state, batch size 64, and weight decay 10^{-4} .

Candidate construction combines single-add and single-delete actions from the five highest-scoring IDs per slot with instantiated constraint heuristics. Before stable deduplication and an 80-candidate cap, each action contributes at most 20 proposal candidates and 30 heuristic candidates. The historical repair is added only to candidate sets used to compute training and validation losses and may contain both an addition and a deletion; inference and candidate-oracle sets are label-blind. The supplementary reproducibility appendix specifies the scoring, ordering, heuristics, chooser architectures, and edge-case conventions.

Candidate-C and Candidate-DP are trained jointly with the Direct-Factor graph encoder and edit heads. Candidate membership is discrete and is constructed from detached proposal logits; gradients then pass through the Candidate-C chooser scores or Candidate-DP’s six selected proposal logits. Candidate-SR instead freezes the proposal model and jointly trains its new graph encoder and reranker. Candidate-C and Candidate-DP compute their candidate-level symbolic terms from role-projected states without alias merging or mandatory insertion of S_i ; Candidate-SR uses the alias-aware reconstruction in Section 7.5. All reported metrics use the latter reconstruction for every system.

7.5 Evaluation protocol

Evaluation reconstructs the pre- and post-edit symbolic states for every test instance, merging graph roles that denote the same entity, ensuring that the base statement is initially present, and

applying deletion before addition. Checkability and satisfaction are then recomputed from these states. Reported metrics use this common procedure for every system. Each aggregate metric is accompanied by its numerator and denominator, and no test metric is used for model selection.

■ **Table 4** Pre-edit state of the primary constraint on the full test split. PFR and EPPF use the checkable-and-violated row as a fixed eligibility set; all 143,316 predictions remain present in the evaluation.

Pre-edit primary state	Instances	Share
Checkable and violated (eligible)	54,234	37.84%
Uncheckable from local evidence	39,781	27.76%
Checkable but already satisfied	49,301	34.40%
Total	143,316	100.00%

PFR and EPPF are evaluated on 54,234 instances because eligibility requires the primary constraint to be checkable and violated before repair. Some test instances lack sufficient local evidence, especially for type- and dependency-related validators, while in others the primary constraint is already satisfied after entity aliases and local evidence are resolved. Neither category represents a primary violation that the predicted edit can meaningfully fix, so both are excluded from the denominator. If an eligible primary becomes uncheckable after the predicted edit, it counts as a failure rather than being removed from the fixed denominator.

8 Results

8.1 Historical fidelity and symbolic behaviour diverge

The aggregate results do not induce a single model ranking. The system that best reproduces historical edits is not the one that most often satisfies the primary validator, and improvements in local satisfaction can coincide with more aggressive evidence removal. We therefore interpret Tables 5–7 as complementary views of repair behaviour rather than interchangeable measures of quality. Arrows in table headers indicate the preferred direction; unmarked columns are descriptive. Bold and underlined values mark the best and second-best results, respectively.

■ **Table 5** Historical fidelity and predicted-edit disruption. Add and Del are the mean numbers of complete addition and deletion operations predicted per instance.

Model	Precision $\uparrow$	Recall $\uparrow$	Micro-F1 $\uparrow$	Add	Del	Disrupt. $\downarrow$
Direct–Passive GNN	0.5659	0.5296	0.5471	0.5159	0.4821	0.9979
Direct–Factor GNN	0.6890	0.6689	0.6788	0.5747	0.4604	1.0352
Candidate–C	0.5318	0.4988	0.5148	0.5676	0.4324	1.0000
Candidate–DP	0.4528	0.4222	0.4370	0.5326	0.4617	<u>0.9943</u>
Candidate–SR	0.2726	0.2557	0.2639	0.0000	1.0000	1.0000
Baseline–DB	0.2897	0.2717	0.2804	0.0000	1.0000	1.0000
Baseline–AM	0.2814	0.0276	0.0503	0.1047	0.0000	0.1047
Baseline–FM	0.2732	0.2580	0.2654	0.5305	0.4766	1.0072
Baseline–DM	<u>0.6563</u>	<u>0.6404</u>	<u>0.6482</u>	0.5603	0.4801	1.0404

At the system level, the Direct–Factor GNN exceeds the Direct–Passive GNN by 0.1317 Micro-F1 while predicting only 0.0373 more operations per instance, so the fidelity difference is not

explained by indiscriminately producing more edits. Baseline-DM is nevertheless only 0.0306 behind the factor model. Concrete constraint definitions therefore carry strong in-distribution priors about curator actions, making this memorisation baseline a much more competitive comparator than the passive GNN. As discussed in Section 9, the Direct-Factor and Direct-Passive systems differ in several respects besides factor execution, so this comparison should be interpreted as system-level rather than as an isolated causal estimate of factorisation.

■ **Table 6** Primary, local, and base-statement-preservation metrics. n_{LocalSat} and n_{Δ} are eligible constraint-occurrence denominators.

Model	PFR $\uparrow$	LocalSat $\uparrow$	n_{LocalSat}	$\Delta\text{LocalSat}$ $\uparrow$	n_{Δ}	BaseDel $\downarrow$	DelBase $\downarrow$	EPPF $\uparrow$	Vacuous $\downarrow$
Direct-Passive GNN	0.8109	0.7622	5,322,054	0.0201	4,691,844	0.4342	0.4345	0.1201	0.3419
Direct-Factor GNN	0.6278	0.7518	5,189,194	0.0154	4,691,910	0.3517	0.3622	0.1053	0.2702
Candidate-C	0.5950	0.7490	5,127,377	0.0156	4,691,615	0.3121	0.3121	0.1026	0.2364
Candidate-DP	0.7707	0.7628	5,331,576	0.0202	4,692,049	0.4442	0.4442	0.0350	0.3576
Candidate-SR	<u>0.9200</u>	0.8021	5,931,955	0.0394	4,689,235	0.9741	0.9741	0.0604	0.5659
Baseline-DB	0.9218	<u>0.7995</u>	5,962,474	0.0349	4,690,179	1.0000	1.0000	0.0000	0.5782
Baseline-AM	0.0148	0.7105	4,692,077	0.0002	4,692,077	0.0000	0.0000	0.0148	0.0000
Baseline-FM	0.6719	0.7530	5,217,600	0.0152	4,691,561	0.3713	0.3714	0.1192	0.2891
Baseline-DM	0.8037	0.7589	5,280,993	0.0185	4,691,750	0.4078	0.4166	0.1462	0.3229

The fidelity advantage does not transfer to symbolic repair outcomes. The Direct-Passive GNN fixes substantially more eligible primaries than the Direct-Factor GNN, but also removes the base statement 8.25 percentage points more often and produces 7.17 points more vacuous improvements. The Direct-Factor GNN therefore preserves the triggering statement more often, yet leaves more primary violations unresolved. Neither system dominates: the passive system obtains stronger validator outcomes partly through more deletion, whereas the factor system achieves higher curator agreement with less evidence removal.

Candidate-DP demonstrates that decision-level penalties can move the targeted rule metrics. Relative to the Direct-Factor GNN, it raises PFR by 14.29 points, raises SIR by 0.93 points, and lowers SRR by 0.09 points. These changes cost 24.18 points of Micro-F1, increase base deletion by 9.25 points, and reduce EPPF by 7.03 points. The extra primary fixes therefore do not constitute a general safety improvement: most of the gain disappears once retention of the base statement is required. Candidate-C occupies a more conservative point, with the lowest base-deletion and vacuous-improvement rates among learned systems, but it improves neither fidelity nor primary repair over the Direct-Factor GNN.

■ **Table 7** Occurrence-pooled secondary satisfaction transitions. The n columns give the eligible constraint-occurrence denominators.

Model	SIR $\uparrow$	n_{SIR}	SRR $\downarrow$	n_{SRR}
Direct-Passive GNN	0.0521	1,305,030	0.0053	3,283,279
Direct-Factor GNN	0.0413	1,305,024	0.0048	3,283,351
Candidate-C	0.0417	1,304,991	0.0041	3,283,089
Candidate-DP	0.0506	1,305,021	<u>0.0039</u>	3,283,493
Candidate-SR	0.1221	1,304,470	0.0074	3,281,230
Baseline-DB	<u>0.1113</u>	1,304,929	0.0095	3,281,715
Baseline-AM	0.0000	1,305,033	0.0000	3,283,509
Baseline-FM	0.0392	1,305,031	0.0049	3,282,995
Baseline-DM	0.0483	1,304,998	0.0060	3,283,217

The secondary transitions reinforce this non-monotonic picture. Candidate-DP achieves the lowest SRR among non-trivial learned systems, but does so while deleting more base statements. Candidate-SR and Baseline-DB obtain the largest SIR values and simultaneously the largest

584 regression rates, consistent with broad deletion changing many local constraint states at once.
 585 SIR and SRR are therefore meaningful only alongside the action and base-statement-preservation
 586 diagnostics.

587 8.2 The aggregate gain is family-dependent

588 The aggregate fidelity advantage of the Direct–Factor GNN over the Direct–Passive GNN is not
 589 uniform across primary constraint families. Supplementary Table S1 reports the family supports,
 590 both direct-system F1 scores, the best system, and Candidate–SR’s base-deletion rate. The
 591 Direct–Factor GNN improves on the passive system in six of nine families, with its largest gain
 592 on **distinct**, but Baseline–DM leads six families overall. Candidate–SR’s deletion shortcut is
 593 more uniform, reaching near-total base deletion in all but **conflictWith**. The aggregate results
 594 should therefore be interpreted as mixtures of family-specific effects rather than evidence that
 595 one architecture improves every rule type.

596 8.3 Local satisfaction admits a deletion shortcut

597 Baseline–DB provides a deterministic counterexample to interpreting high satisfaction as useful
 598 repair: deleting S_i in every instance yields PFR 0.9218 and Local Satisfaction 0.7995, while
 599 EPPF is necessarily zero. Candidate–SR independently learns nearly the same strategy from a
 600 satisfaction-only objective. It predicts one deletion in every case, removes S_i 97.41% of the time,
 601 and attains PFR 0.9200 and Local Satisfaction 0.8021, despite a historical-fidelity Micro-F1 of
 602 0.2639. The result does not imply that deletion is intrinsically wrong. It establishes that local
 603 satisfaction cannot distinguish justified deletion from systematic removal of the evidence that
 604 made the constraint applicable.

605 8.4 Executable factors are used, principally through secondary context

606 The two mechanistic diagnostics answer different questions. Satisfaction probes test whether
 607 factor states encode the symbolic satisfaction labels; pressure masking tests whether factor-to-
 608 local messages affect the decoded action. Supplementary Table S2 reports support, class balance,
 609 discrimination, and calibration for every factor family. The pre-repair heads show strong label-
 610 aligned signal on average, but support and class balance vary substantially across families, and
 611 calibration is uneven. High aggregate F1 therefore does not establish uniform rare-violation dis-
 612 crimination or agreement with independently reconstructed constraint status. The post-historical-
 613 edit head is used only during training and validation and is not included in this diagnostic.

■ **Table 8** Direct–Factor GNN inference-time pressure masking results.

Masking intervention	Δ Micro-F1 $\uparrow$	Δ PFR $\uparrow$	Δ LocalSat $\uparrow$	Δ Disruption $\downarrow$	Prediction changed
No factor pressure	-0.3985	-0.5818	-0.0422	-0.1268	0.5855
Primary-only pressure	<u>-0.3422</u>	<u>-0.4122</u>	<u>-0.0316</u>	<u>-0.1224</u>	0.5372
Secondary-only pressure	-0.0024	-0.0036	0.0002	-0.0004	0.0295

614 Masking establishes that the trained Direct–Factor model materially uses the factor pathway.
 615 Masking all pressure changes 58.55% of predictions and lowers Micro-F1 by 0.3985. More strik-
 616 ingly, retaining only secondary pressure nearly reproduces normal inference, whereas retaining
 617 only primary pressure loses 0.3422 F1. Non-primary factors therefore carry most of the pressure-
 618 dependent imitation behaviour. Because the model weights are fixed, this intervention establishes
 619 pathway use rather than validator-equivalent reasoning, semantic causality, or a safety benefit; it
 620 also does not isolate factor execution as the cause of the between-system fidelity difference.

8.5 Base-statement-preserving candidates are the main bottleneck

Candidate-based systems can fail because their generator does not propose a suitable repair or because their decision rule overlooks one that is available. We distinguish these failure modes by inspecting the label-blind candidate sets after inference. Availability records whether a set contains an edit that fixes the primary constraint without causing a secondary regression or reducing satisfaction among constraints checkable both before and after the edit; selection records whether the system chooses such an edit. We repeat the analysis while additionally requiring preservation of the base statement.

The sets contain about 42.4 candidates on average. Supplementary Table S3 identifies availability and selection for the Direct-Factor proposal and the three candidate-based systems over both the full test set and the 54,234 PFR-eligible cases. Among eligible cases, 90.78–91.19% of sets contain a suitable candidate, whereas only 24.85–25.85% contain one that also preserves S_i . Selection of such a base-statement-preserving candidate ranges from 3.16% to 9.57%. Thus ordinary primary-fixing, no-regression candidates are commonly generated; base-statement-preserving generation and selection are the bottlenecks. These are candidate-set diagnostics, not aggregate repair-quality measures.

9 Discussion

What executable factors contribute.

At the system level, the Direct-Factor GNN achieves higher historical-repair fidelity than the Direct-Passive GNN, while masking pressure with all learned parameters held fixed shows that predictions of the trained Direct-Factor model materially depend on that pathway. These observations are complementary but should not be conflated: because the two systems differ in capacity, depth, dropout, graph content, and auxiliary objectives, the 0.1317 Micro-F1 gap does not isolate the causal effect of factorisation. What the masking experiment does establish is functional use of the explicit factor pathway. The near-complete recovery obtained with secondary-only pressure further indicates that the model uses constraints as a representation of the wider local repair context, rather than treating only the originally flagged rule as relevant. This is a useful form of inspectability: factor states can be probed and their messages intervened upon. It is not, however, evidence that the neural executors implement the associated validators or that factor use improves safety. The strong Baseline-DM result and imperfect calibration of the factor heads counsel a narrower claim: the executable interface provides semantically designated channels whose states align with symbolic satisfaction labels and whose messages influence repair prediction, against a benchmark with powerful in-distribution definition priors.

Why fidelity and symbolic safety must remain separate.

Historical fidelity measures agreement with one observed curator action; local validators measure the consequences of an action within bounded evidence. Neither is a complete gold standard. A historically plausible repair may retain a violation or create a neighbouring one, while a validator-improving repair may simply delete the statement that exposed the problem. Candidate-SR makes the latter failure mode explicit: a learned optimiser can discover satisfaction-by-deletion without being given an always-delete rule. Base retention and EPPF reveal this shortcut, but they too are diagnostics rather than universal correctness criteria, because deletion can be the appropriate repair. The practical implication is to report a profile of fidelity, primary repair, secondary transitions, disruption, and base-statement preservation, or a Pareto frontier over them, instead of collapsing them into a single “safe repair” score.

665 Implications for repair generation and control.

666 Candidate-DP shows that candidate-level symbolic terms can improve primary and secondary
 667 validator outcomes, but the accompanying losses in fidelity and EPPF show that the objective is
 668 under-specified. Preservation cannot be repaired solely by reranking when only about one quarter
 669 of PFR-eligible instances has a base-statement-preserving option satisfying the local requirements.
 670 Future systems should first broaden structured generation toward witness-specific alternatives—
 671 for example, adding missing support, replacing a value, or editing conflicting context—and then
 672 compare complete actions with symbolic revalidation. Base deletion can be retained as an admiss-
 673 ible operation, but should require separate evidence or an explicit policy rather than receiving
 674 an automatic advantage from making constraints disappear. Abstention and curator review are
 675 preferable when no candidate meets the chosen criteria.

676 Limitations and threats to validity.

677 The symbolic analysis is local. Only 37.84% of test instances have a primary constraint that is
 678 both checkable and violated in the reconstructed pre-edit state, and validators use a bounded
 679 one-hop graph without a materialised transitive class hierarchy. Reported satisfaction therefore
 680 describes consequences under the available local evidence, not global Wikidata validity. An
 681 operation outside the represented subject and local-predicate closure is a symbolic no-op under
 682 this evaluator, and its frequency is not quantified here. The repair language is also restricted to
 683 at most one direct-statement addition and deletion: qualifier, rank, reference, and T-box repairs
 684 are excluded, while base retention captures only one aspect of evidence preservation.

685 Factor-head supervision and reported symbolic outcomes use different state constructions. The
 686 states used to generate the supervision labels do not merge coincident entity roles or guarantee the
 687 initial presence of S_i , whereas evaluation resolves aliases, inserts S_i , and applies deletion before
 688 addition. This mismatch may change checkability for some occurrences and introduce semantic
 689 drift. All factor-supervised systems share the same labels, preserving their comparability, but
 690 future datasets should use the evaluation state construction for both supervision and reporting.
 691 Accordingly, the satisfaction probes establish alignment with the symbolic satisfaction labels
 692 rather than agreement with independently reconstructed constraint status.

693 Causal claims about the candidate systems are limited by joint changes to objectives and
 694 decision mechanisms. The reported learned systems use one selected run, so small differences
 695 should not be read as estimates of variability.

696 Finally, the random split permits entities and concrete constraint definitions to recur across
 697 train and test, particularly given node-ID encoding. As exposed by Baseline-DM, the evaluation
 698 therefore measures same-corpus, in-distribution historical event prediction rather than temporal,
 699 definition-disjoint, or unseen-entity transfer. Temporal and definition-disjoint splits, together
 700 with multi-seed evaluation, are needed to assess generalisation beyond these priors. The finite
 701 proposal and heuristic candidate sets also bound the candidate-oracle analysis, and historical
 702 edits remain observational supervision that may reflect ambiguity, incomplete context, or curator
 703 preference rather than a unique correct repair.

704 10 Conclusion

705 We introduced executable constraint factors for neural Wikidata repair and evaluated them with
 706 separate measures of curator agreement, post-edit constraint behaviour, collateral transitions,
 707 disruption, and base-statement preservation. Among the evaluated systems, the Direct-Factor
 708 GNN attains the highest historical fidelity, and pressure-masking interventions on the trained

model show that its predictions materially depend on factor pressure, primarily from non-primary constraint context. These observations support the value of the executable interface for representation and inspectability, but they do not by themselves establish the isolated causal effect of factorisation or symbolic safety. Candidate-level penalties improve selected validator outcomes at a large cost in fidelity and base-statement-preserving repair, while a satisfaction-only reranker learns to delete the base statement almost universally.

The central lesson is therefore methodological as well as representational: making rules executable and inspectable does not make a learned repair useful by construction. Reliable systems need preservation-aware candidate generation, explicit comparison of complete edits, richer repair actions, and evaluation under multiple non-substitutable criteria. Multi-seed, temporal, and definition-disjoint tests, consistent reconstruction for supervision and evaluation, broader symbolic evidence, and curator-facing abstention are necessary next steps toward repair models that are both faithful to collaborative practice and defensible under rule-based validation.

References

- Shqiponja Ahmetaj, Robert David, Magdalena Ortiz, Axel Polleres, Bojken Shehu, and Mantas Šimkus. Reasoning about explanations for non-validation in shacl. In *Proceedings of the 18th International Conference on Principles of Knowledge Representation and Reasoning*, pages 12–21, 2021. doi:10.24963/kr.2021/2.
- Shqiponja Ahmetaj, Robert David, Axel Polleres, and Mantas Šimkus. Repairing shacl constraint violations using answer set programming. In *The Semantic Web – ISWC 2022*, volume 13489 of *Lecture Notes in Computer Science*, pages 375–391, 2022. doi:10.1007/978-3-031-19433-7_22.
- Shqiponja Ahmetaj, Robert David, Axel Polleres, and Mantas Šimkus. A logic programming approach to repairing SHACL constraint violations. *Transactions on Graph Data and Knowledge*, 3(3):1:1–1:36, 2025. URL: <https://drops.dagstuhl.de/entities/document/10.4230/TGDK.3.3.1>, doi:10.4230/TGDK.3.3.1.
- Leopoldo Bertossi. *Database Repairing and Consistent Query Answering*. Springer, 2022. doi:10.1007/978-3-031-01883-1.
- Fredo Erxleben, Michael Günther, Markus Krötzsch, Julian Mendez, and Denny Vrandečić. Introducing wikidata to the linked data web. In *The Semantic Web – ISWC 2014*, volume 8796 of *Lecture Notes in Computer Science*, pages 50–65, 2014. doi:10.1007/978-3-319-11964-9_4.
- Nicolas Ferranti, Jairo Francisco de Souza, Shqiponja Ahmetaj, and Axel Polleres. Formalizing and validating wikidata’s property constraints using shacl and sparql. *Semantic Web*, 15(6):2333–2380, 2024. doi:10.3233/SW-243611.
- Nicolas Ferranti, Dayane Guimarães, Jairo Francisco de Souza, and Axel Polleres. Formalizing repairs for wikidata constraint violations: A taxonomy and empirical analysis. In *The Semantic Web – ISWC 2025*, pages 369–387, 2025. doi:10.1007/978-3-032-09527-5_20.
- Marc Fischer, Mislav Balunovic, Dana Drachslers-Cohen, Timon Gehr, Ce Zhang, and Martin Vechev. DL2: Training and querying neural networks with logic. In *Proceedings of the 36th International Conference on Machine Learning*, pages 1931–1941, 2019. URL: <https://proceedings.mlr.press/v97/fischer19a.html>.
- Victor García Satorras, Emiel Hoogeboom, and Max Welling. Neural enhanced belief propagation on factor graphs. In *Proceedings of The 24th International Conference on Artificial Intelligence and Statistics*, 2021. URL: <https://proceedings.mlr.press/v130/garcia-satorras21a.html>.
- Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured nlp tasks without finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 10932–10952, 2023. doi:10.18653/v1/2023.emnlp-main.674.
- Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, Jose Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan F. Sequeda, Steffen Staab, and Antoine Zimmermann. Knowledge graphs. *ACM Computing Surveys*, 54(4):71, 2021. doi:10.1145/3447772.
- Weihua Hu, Bowen Liu, Joseph Gomes, Marinka Zitnik, Percy Liang, Vijay Pande, and Jure Leskovec. Strategies for pre-training Graph Neural Networks. In *International Conference on Learning Representations*, 2020. URL: <https://openreview.net/forum?id=HJ1WWJSFDH>.
- Nan Jiang, Yexiang Xue, Hedayat Khosravi, Teague Vossen, and Willem-Jan van Hoeve. Constraint reasoning embedded structured prediction. *Journal of Machine Learning Research*, 23, 2022. URL: <https://jmlr.org/papers/v23/21-1484.html>.
- Holger Knublauch and Dimitris Kontokostas. Shapes constraint language (shacl). W3C Recommendation, 2017. URL: <https://www.w3.org/TR/shacl/>.
- Jose Emilio Labra Gayo. Wshex: A language to describe and validate wikibase entities. In *Proceedings of the 3rd Wikidata Workshop 2022*,

2022. URL: <https://ceur-ws.org/Vol-3262/paper3.pdf>, doi:10.48550/arXiv.2208.02697.
- 16 David L. Martin and Peter F. Patel-Schneider. Wikidata constraints on mars, 2020. URL: <https://arxiv.org/abs/2008.03900>, doi:10.48550/arXiv.2008.03900.
- 17 Maximilian Marx, Markus Krötzsch, and Veronika Thost. Logic on mars: Ontologies for generalised property graphs. In *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence*, pages 1188–1194, 2017. doi:10.24963/ijcai.2017/165.
- 18 MediaWiki contributors. Extension:wikibasequalityconstraints, 2025. Accessed 2026-06-09. URL: <https://www.mediawiki.org/wiki/Extension:WikibaseQualityConstraints>.
- 19 Heiko Paulheim. Knowledge graph refinement: A survey of approaches and evaluation methods. *Semantic Web*, 8(3):489–508, 2017. doi:10.3233/SW-160218.
- 20 Thomas Pellissier Tanon. Wikidata constraint violations – july 2018 – extended, 2020. Dataset, CC0. doi:10.6084/m9.figshare.13338743.
- 21 Thomas Pellissier Tanon, Camille Bourgaux, and Fabian Suchanek. Learning how to correct a knowledge base from the edit history. In *The World Wide Web Conference*, pages 1465–1475, 2019. doi:10.1145/3308558.3313584.
- 22 Thomas Pellissier Tanon and Fabian Suchanek. Neural knowledge base repairs. In *The Semantic Web – ESWC 2021*, pages 287–303, 2021. doi:10.1007/978-3-030-77385-4_17.
- 23 Alessandro Piscopo and Elena Simperl. Who models the world? collaborative ontology creation and user roles in wikidata. *Proceedings of the ACM on Human-Computer Interaction*, 2(CSCW), 2018. doi:10.1145/3274410.
- 24 Axel Polleres, Romana Pernisch, Angela Bonifati, Stefania Dumbrava, Mikhail Galkin, Luis Hampel, Markus Krötzsch, Albert Meroño-Peñuela, Heiko Paulheim, Anisa Rula, Chris Russell, Daniel Schwabe, Juan Sequeda, Marieke van Erp, Maria-Esther Vidal, and Marina Vos. How does knowledge evolve in open knowledge graphs? *Transactions on Graph Data and Knowledge*, 1(1):7:1–7:60, 2023. doi:10.4230/TGDK.1.1.11.
- 25 Lukas Schmelzeisen, Corina Dima, and Steffen Staab. Wikidated 1.0: An evolving knowledge graph dataset of wikidata’s revision history. In *Proceedings of the 2nd Wikidata Workshop*, 2021. URL: <https://ceur-ws.org/Vol-2982/paper-11.pdf>.
- 26 Shape Expressions Community Group. Shape expressions language 2.1, 2019. URL: <https://shex.io/shex-semantic/>.
- 27 Kartik Shenoy, Filip Ilievski, Daniel Garijo, Daniel Schwabe, and Pedro Szekely. A study of the quality of wikidata. *Journal of Web Semantics*, 72:100679, 2022. doi:10.1016/j.websem.2021.100679.
- 28 Budhitama Subagdja, D. Shanthoshigaa, Zhaoxia Wang, and Ah-Hwee Tan. Machine learning for refining knowledge graphs: A survey. *ACM Computing Surveys*, 56(6):156, 2024. doi:10.1145/3640313.
- 29 Nicolas Troquard, Roberto Confalonieri, Pietro Galliani, Rafael Peñaloza, Daniele Porello, and Oliver Kutz. Repairing ontologies via axiom weakening. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.
- 30 Miguel Vázquez, Kevin Innerebner, Alexander Prock, Günter Klambauer, Elisabeth Lex, Johannes Schimunek, and Axel Polleres. Structure is the signal: Graph encodings and gnn for constraint repair in collaborative kgs. In *The Semantic Web – ESWC 2026*, volume 16549 of *Lecture Notes in Computer Science*, pages 278–296, 2026. doi:10.1007/978-3-032-25156-5_15.
- 31 Denny Vrandečić and Markus Krötzsch. Wikidata: A free collaborative knowledgebase. *Communications of the ACM*, 57(10):78–85, 2014. doi:10.1145/2629489.
- 32 Wikibase contributors. Wikibase constraints documentation, 2026. Accessed 2026-06-09. URL: https://doc.wikimedia.org/Wikibase/master/php/docs_topics_constraints.html.
- 33 Wikidata contributors. Help:property constraints portal, 2026. Accessed 2026-06-09. URL: https://www.wikidata.org/wiki/Help:Property_constraints_portal.
- 34 Wikimedia contributors. Wikidata:data model, 2026. Accessed 2026-06-09. URL: https://www.wikidata.org/wiki/Wikidata:Data_model.
- 35 Jingyi Xu, Guy van den Broeck, and Adnan Darwiche. A semantic loss function for deep learning with symbolic knowledge. In *Proceedings of the 35th International Conference on Machine Learning*, 2018. URL: <https://proceedings.mlr.press/v80/xu18h.html>.
- 36 Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are Graph Neural Networks? In *International Conference on Learning Representations*, 2019. URL: <https://openreview.net/forum?id=ryGs6iA5Km>.
- 37 Zhun Yang, Joohyung Lee, and Chiyoun Park. Injecting logical constraints into neural networks via straight-through estimators. In *Proceedings of the 39th International Conference on Machine Learning*, 2022. URL: <https://proceedings.mlr.press/v162/yang22h.html>.
- 38 Qinggang Zhang, Junnan Dong, Keyu Duan, Xiao Huang, Yezi Liu, and Linchuan Xu. Contrastive knowledge graph error detection. In *Proceedings of the 31st ACM International Conference on Information and Knowledge Management*, pages 2590–2599, 2022. doi:10.1145/3511808.3557264.
- 39 Zhen Zhang, Mohammed Haroon Dupty, Fan Wu, Javen Qinfeng Shi, and Wee Sun Lee. Factor graph neural networks. *Journal of Machine Learning Research*, 24, 2023. URL: <https://www.jmlr.org/papers/v24/21-0434.html>.